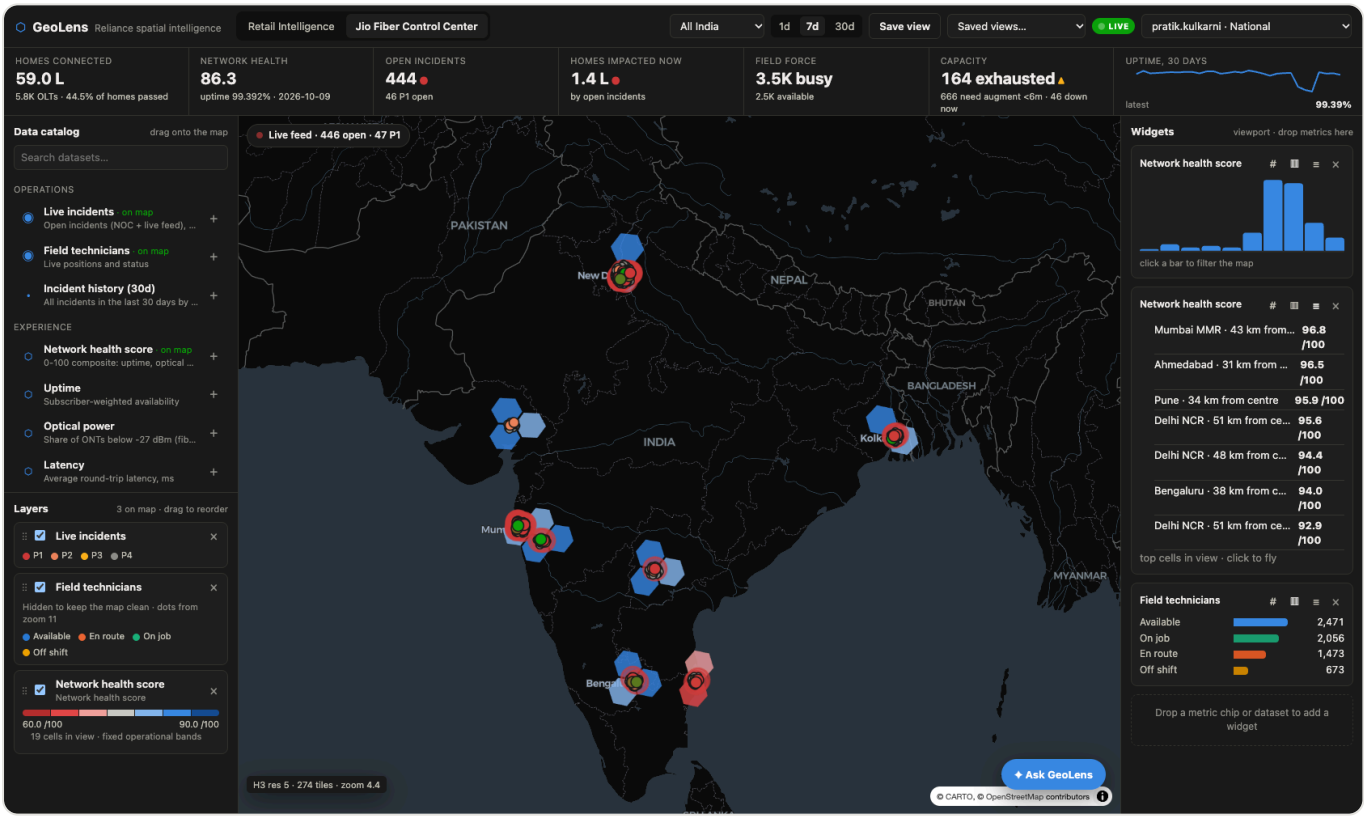
SOLUTION OVERVIEW · JIO FIBER HOME

GeoLens: a spatial Network Control Center for Jio Fiber, built on Databricks

One live map of network health, incidents, field force and capacity across eight metros. Served from Lakebase in milliseconds, governed by Unity Catalog, and delivered as a Databricks App that operations teams use with drag and drop.



CONTENTS

1. Executive summary
2. The operational challenge
3. The data we modelled
4. Solution architecture
5. Inside Lakebase
6. Security and access control
7. The storyline: a day in the NOC
8. Always current: data freshness
9. End-to-end test results
10. Scale and performance
Appendix A. Metric definitions
Appendix B. Glossary
Appendix C. Technology stack
Appendix D. Basemap: removing the CARTO dependency

1. Executive summary

Jio Fiber operations teams need a single, trusted view of how the network is performing for customers, where it is failing right now, and where the field force should go next. GeoLens provides that view as an interactive map built on the Databricks Data Intelligence Platform, in the spirit of commercial location-intelligence tools, but running entirely on the customer's own lakehouse data and governance.

5.9 M	5,841	~13 ms	< 1 min
subscriber homes modelled across 8 metros	OLTs and 24,261 FDH cabinets on the map	median server time per map tile (p95 35 ms)	from a Delta table change to every open map, no reload

- **See the whole network at a glance.** H3 hexagons colour every area of every metro by a 0 to 100 network health score built from uptime, optical margin, latency, loss, throughput and trouble tickets. Fixed operational bands keep healthy areas calm and make real problems stand out.
- **Act on live incidents.** Open incidents appear on the map from the first screen, coloured by severity. Critical (P1) incidents keep glowing until they are resolved. One click shows impact, root cause and the nearest available technicians, and dispatches one.
- **Drill without losing context.** Click any hexagon to zoom into exactly that area, then keep clicking down to street level. A breadcrumb and a Back button return you to where you were.
- **Build views with drag and drop.** A data catalog of 17 layers (incidents, technicians, health, uptime, optical power, capacity, growth and cross-sell) can be dragged onto the map or into widgets.
- **Governed by design.** Region and metro entitlements are defined once in Unity Catalog and enforced both by Unity Catalog row filters and by Postgres row-level security in Lakebase. A Delhi NOC lead only ever sees Delhi.
- **Ask in plain language.** An AI assistant built on Databricks Foundation Model APIs answers questions such as "which OLTs will run out of capacity first?" and moves the map to show the answer.

2. The operational challenge

A home broadband network at Jio's scale generates signals from many systems: network management alarms, ONT optical telemetry, OLT port utilisation, trouble tickets, workforce positions and the subscriber base. Each team typically sees one slice in one tool. The questions the NOC and regional leadership actually ask are spatial and cross-cutting:

Persona	Questions they need answered in seconds
NOC lead	What is broken right now, how many homes are affected, which incidents are critical, and who is the nearest available technician?
Regional head	Which localities in my region have degrading customer experience this week, and is it optical, capacity or field related?
Capacity planner	Which OLTs are exhausted or will exhaust within six months, and where should we augment or split PON ports?
Field operations	Where are my technicians, who is free, and are dispatches matching incident severity?
Growth and sales	Where are homes passed but not connected, where is premium plan mix low, and where are Reliance Retail customers who do not yet have Jio Fiber?

The solution therefore had to meet five requirements that were set at the start:

- **Hexagon maps** of network KPIs at multiple resolutions, from national down to street level.
- **Fast responses** for thousands of concurrent users, so the serving layer must be an operational database (Lakebase), not a warehouse.
- **Live updates**: the map must refresh by itself as new data lands, with no page reload.
- **Region-scoped access** managed centrally through Unity Catalog and Postgres row-level security.
- **Drag and drop**: adding information to the map should not require code or SQL.

3. The data we modelled

To demonstrate behaviour at realistic scale without using customer data, we generated a synthetic but internally consistent Jio Fiber network for eight metros: Mumbai MMR, Pune and Ahmedabad (West), Delhi NCR (North), Bengaluru, Hyderabad and Chennai (South), and Kolkata (East). Every dataset is keyed to the same geography so that layers line up exactly on the map.

Dataset	What it represents	Grain	Volume
Demographic grid	Population and households on a fine hexagon grid, the base for coverage and penetration	H3 res 9 cell	160,148 cells · 72.4 M people
OLTs	Optical Line Terminals with vendor model, PON ports, split ratio, homes passed and connected, utilisation, capacity status	OLT	5,841
FDH cabinets	Fiber distribution hubs (splitter cabinets) with 7-day health	Cabinet	24,261
Subscribers	Connected homes with plan, tenure and serving OLT and FDH	Home	5,904,185
Homes passed	Homes within reach of the fiber network	OLT area	13.3 M
Fiber routes	Feeder and distribution cable segments as line geometry, with 30-day cut counts	Segment	30,102
Network incidents	Fiber cuts, LOS alarm clusters, power failures, FDH damage, card failures, planned work; severity P1 to P4, homes impacted, root cause	Incident	25,605 (30 days)
Trouble tickets	Customer complaints linked to homes and incidents	Ticket	318,874
Technicians	Field force with skill, shift, status and live position	Technician	6,673
OLT hourly telemetry	Port utilisation, uplink utilisation, alarms and status per hour	OLT × hour	4.24 M rows
Network experience, daily	Health score, uptime, optical power, latency, loss, throughput, tickets, incidents per hexagon per day, rolled up at five resolutions	H3 res 5 to 9 × day	5.23 M rows
Daily KPIs	Metro-level network KPIs for trend lines	Metro × day	240
Access entitlements	Which person or role may see which region or metro	Principal × scope	12 rules

Realism built into the data

- **Thirty days of history** (10 September to 9 October 2026) with daily and hourly patterns, so trends, weekly comparisons and sparklines behave like production data.
- **Network vintage matters.** Each OLT area has a build year. Older areas show weaker optical margins and more faults, which is how real fiber estates age.
- **Named stress events** make the story visible: a three-day cyclone in Chennai drives a cluster of power failures and cable cuts, and construction activity in Gurugram drives repeated distribution cuts.

- **Correlated signals.** Cell-level noise is blended with OLT-level effects, so a failing OLT degrades every hexagon it serves, as in a real PON tree.
- **Capacity pressure** is concentrated in dense localities, producing exhausted OLTs and "augment within six months" candidates for the planner.
- **Live operations.** In this environment a simulator inside the app raises new incidents every few seconds, moves technicians towards jobs and resolves incidents. In production the same tables are fed by the network management system, OSS alarms and the workforce management system.

> All data in this document is synthetic and was created for the demonstration. Table structures, grain and metric definitions are designed to map directly to Jio's real NMS, OSS, BSS and workforce sources.

4. Solution architecture

The design keeps one copy of the truth in the lakehouse and pushes a serving copy into Lakebase, the Databricks managed Postgres database, for low-latency map queries. Governance is defined once in Unity Catalog and enforced at both layers.

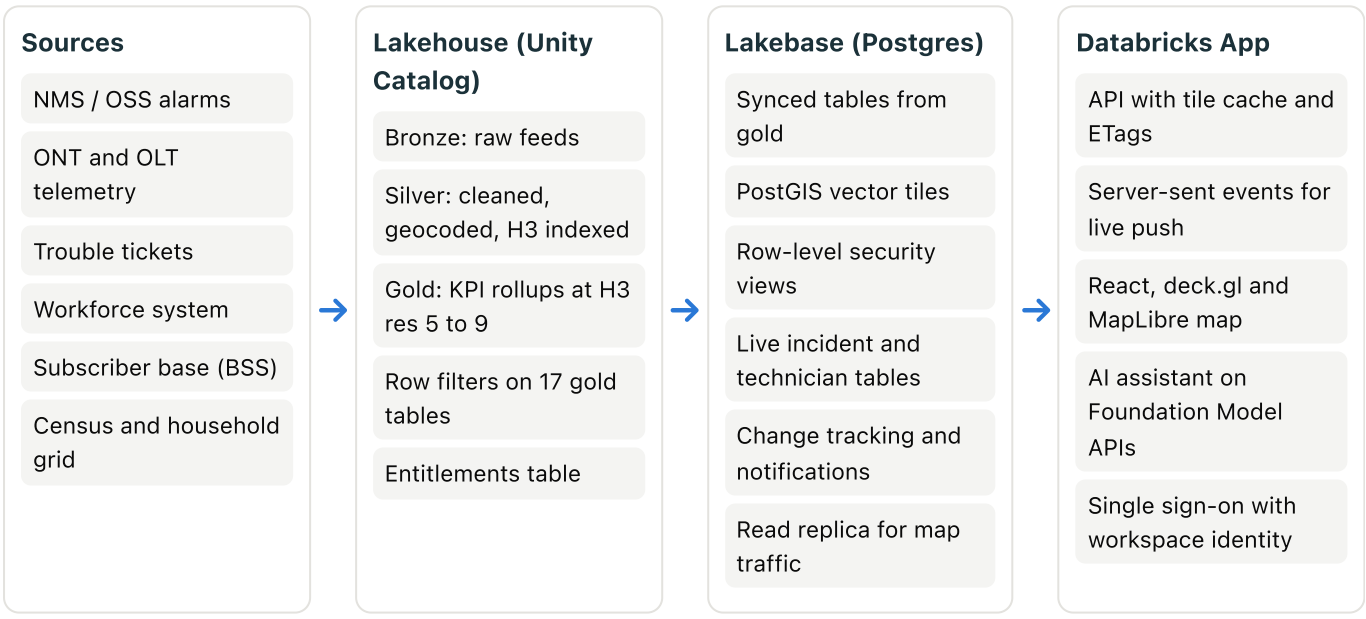


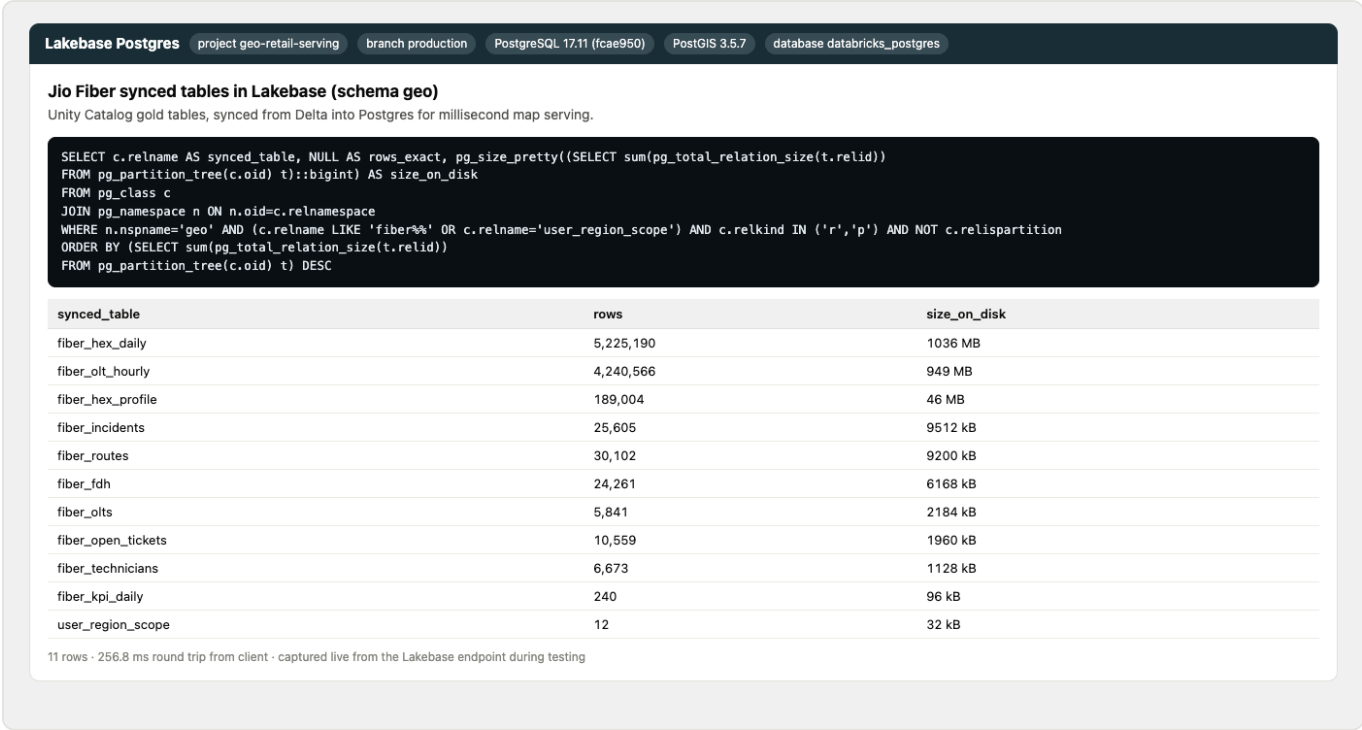
Figure 1. Data flows left to right. Entitlements flow from Unity Catalog into Lakebase so both layers enforce the same scope.

Key design choices

Choice	Why
H3 hexagons at resolutions 5 to 9	Equal-area cells that aggregate cleanly. The map picks a resolution from the zoom level and requests fixed tiles of at most 343 cells, so every request is small and cacheable.
Lakebase as the serving layer	Map interactions need single-digit to tens of milliseconds per request and thousands of concurrent users. Lakebase gives Postgres latency and concurrency, with tables synced from Unity Catalog instead of hand-built ETL.
Read replica for map traffic	Tile and point reads go to a read-only endpoint; writes (dispatches, saved views) go to the primary.
PostGIS vector tiles	Fiber routes are line geometry. PostGIS renders them as vector tiles on demand, so tens of thousands of cable segments stay fast.
Push, not poll	Postgres notifications drive server-sent events to every open browser. Only the layers whose data changed are refetched.
One entitlement model	The same region and metro scope table drives Unity Catalog row filters for analysts and Postgres row-level security for app users.

5. Inside Lakebase

The screenshots below were captured live from the Lakebase endpoint during end-to-end testing. They show the synced Jio Fiber tables, sample rows the map reads, the live operational tables the app writes, and the change tracking that drives automatic refresh.



Lakebase Postgres project geo-retail-serving branch production PostgreSQL 17.11 (fcae950) PostGIS 3.5.7 database databricks_postgres

Jio Fiber synced tables in Lakebase (schema geo)

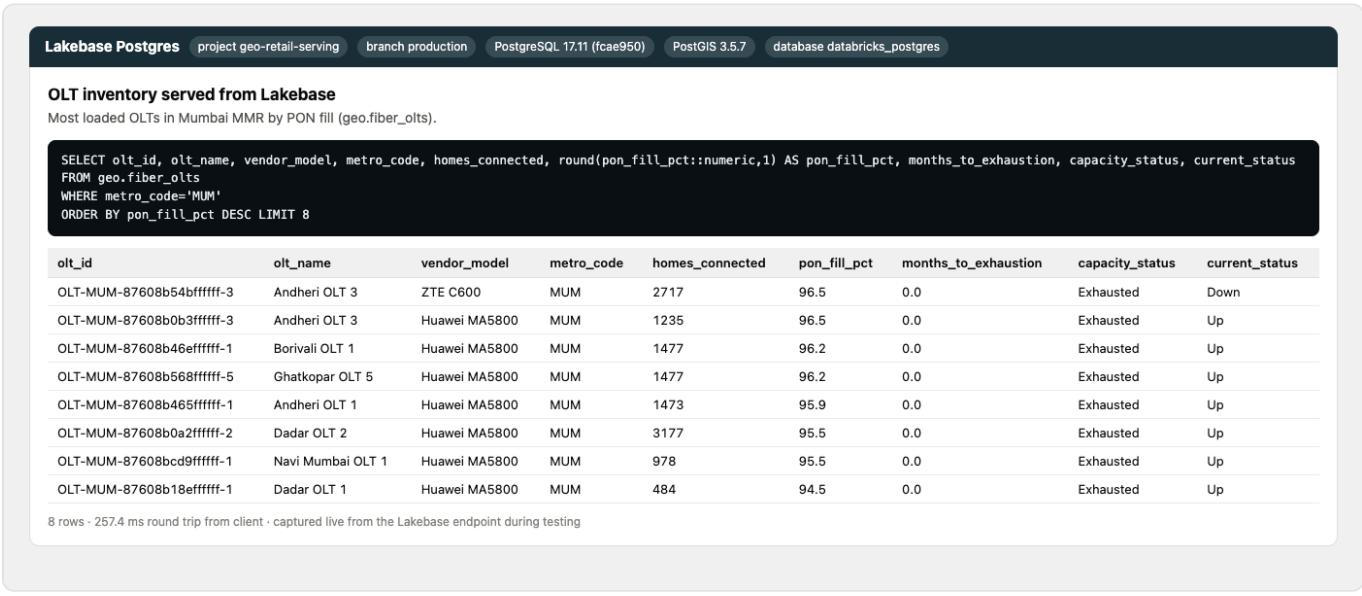
Unity Catalog gold tables, synced from Delta into Postgres for millisecond map serving.

```
SELECT c.relname AS synced_table, NULL AS rows_exact, pg_size_pretty((SELECT sum(pg_total_relation_size(t.relid))
FROM pg_partition_tree(c.oid) t)::bigint) AS size_on_disk
FROM pg_class c
JOIN pg_namespace n ON n.oid=c.relnamespace
WHERE n.nspname='geo' AND (c.relname LIKE 'fiber%%' OR c.relname='user_region_scope') AND c.relkind IN ('r','p') AND NOT c.relispartition
ORDER BY (SELECT sum(pg_total_relation_size(t.relid))
FROM pg_partition_tree(c.oid) t) DESC
```

synced_table	rows	size_on_disk
fiber_hex_daily	5,225,190	1036 MB
fiber_olt_hourly	4,240,566	949 MB
fiber_hex_profile	189,004	46 MB
fiber_incidents	25,605	9512 kB
fiber_routes	30,102	9200 kB
fiber_fdh	24,261	6168 kB
fiber_olts	5,841	2184 kB
fiber_open_tickets	10,559	1960 kB
fiber_technicians	6,673	1128 kB
fiber_kpi_daily	240	96 kB
user_region_scope	12	32 kB

11 rows · 256.8 ms round trip from client · captured live from the Lakebase endpoint during testing

Figure 2. Jio Fiber tables synced from Unity Catalog gold into Lakebase, with exact row counts. Row counts match the Delta source table for table.



Lakebase Postgres project geo-retail-serving branch production PostgreSQL 17.11 (fcae950) PostGIS 3.5.7 database databricks_postgres

OLT inventory served from Lakebase

Most loaded OLTs in Mumbai MMR by PON fill (geo.fiber_olts).

```
SELECT olt_id, olt_name, vendor_model, metro_code, homes_connected, round(pon_fill_pct::numeric,1) AS pon_fill_pct, months_to_exhaustion, capacity_status, current_status
FROM geo.fiber_olts
WHERE metro_code='MUM'
ORDER BY pon_fill_pct DESC LIMIT 8
```

olt_id	olt_name	vendor_model	metro_code	homes_connected	pon_fill_pct	months_to_exhaustion	capacity_status	current_status
OLT-MUM-87608b54bffffff-3	Andheri OLT 3	ZTE C600	MUM	2717	96.5	0.0	Exhausted	Down
OLT-MUM-87608b0b3ffffff-3	Andheri OLT 3	Huawei MA5800	MUM	1235	96.5	0.0	Exhausted	Up
OLT-MUM-87608b46effffff-1	Borivali OLT 1	Huawei MA5800	MUM	1477	96.2	0.0	Exhausted	Up
OLT-MUM-87608b568ffffff-5	Ghatkopar OLT 5	Huawei MA5800	MUM	1477	96.2	0.0	Exhausted	Up
OLT-MUM-87608b465ffffff-1	Andheri OLT 1	Huawei MA5800	MUM	1473	95.9	0.0	Exhausted	Up
OLT-MUM-87608b0a2ffffff-2	Dadar OLT 2	Huawei MA5800	MUM	3177	95.5	0.0	Exhausted	Up
OLT-MUM-87608bcd9ffffff-1	Navi Mumbai OLT 1	Huawei MA5800	MUM	978	95.5	0.0	Exhausted	Up
OLT-MUM-87608b18effffff-1	Dadar OLT 1	Huawei MA5800	MUM	484	94.5	0.0	Exhausted	Up

8 rows · 257.4 ms round trip from client · captured live from the Lakebase endpoint during testing

Figure 3. OLT inventory as served to the map: the most loaded Mumbai OLTs, already exhausted and flagged for augmentation.

Lakebase Postgres project geo-retail-serving | branch production | PostgreSQL 17.11 (fcae950) | PostGIS 3.5.7 | database databricks_postgres

H3 network-health cells served from Lakebase

Weakest res-8 Mumbai MMR cells on the latest day (geo.fiber_hex_daily).

```
SELECT to_hex(h3) AS h3_cell, res, metro_code, day, round(health_score::numeric,1) AS health, round(uptime_pct::numeric,2) AS uptime_pct, subscribers
FROM geo.fiber_hex_daily
WHERE res=8 AND metro_code='MUM' AND day='2026-10-09'
ORDER BY health_score LIMIT 8
```

h3_cell	res	metro_code	day	health	uptime_pct	subscribers
88608b5099fffff	8	MUM	2026-10-09	28.3	89.72	669
88608bcf41fffff	8	MUM	2026-10-09	32.3	95.82	48
88608bcf55fffff	8	MUM	2026-10-09	33.4	67.29	135
88608b0a21fffff	8	MUM	2026-10-09	33.7	74.86	3250
88608b5567fffff	8	MUM	2026-10-09	34.3	92.92	1681
88608bcf5dfffff	8	MUM	2026-10-09	34.8	93.40	111
88608b540bfffff	8	MUM	2026-10-09	35.3	92.81	1560
88608b5689fffff	8	MUM	2026-10-09	37.0	92.97	1317

8 rows · 281.6 ms round trip from client · captured live from the Lakebase endpoint during testing

Figure 4. Daily network experience per H3 hexagon. The map requests these by tile and colours each cell by health score.

Lakebase Postgres project geo-retail-serving | branch production | PostgreSQL 17.11 (fcae950) | PostGIS 3.5.7 | database databricks_postgres

Live operational tables (written by the app)

Most recent live incidents, updated every few seconds (app.live_incidents).

```
SELECT incident_id, severity, incident_type, metro_code, status, homes_impacted, assigned_technician, to_char(start_ts,'HH24:MI:SS') AS started
FROM app.live_incidents
ORDER BY start_ts DESC LIMIT 8
```

incident_id	severity	incident_type	metro_code	status	homes_impacted	assigned_technician	started
LIVE-261011151031-7006	P1	Feeder fiber cut	CHE	Open	1846		15:10:31
LIVE-261011151027-1073	P3	LOS alarm cluster	MUM	Open	68		15:10:27
LIVE-261011151015-9513	P1	Power failure	KOL	Open	1915	TECH-KOL-0345	15:10:15
LIVE-261011151011-5182	P3	Distribution fiber cut	MUM	Open	118	TECH-MUM-0962	15:10:11
LIVE-261011151007-7762	P3	Distribution fiber cut	MUM	Open	172	TECH-MUM-0854	15:10:07
LIVE-261011151007-2704	P3	FDH damage	MUM	Open	81	TECH-MUM-0518	15:10:07
LIVE-261011151003-6431	P3	Distribution fiber cut	DEL	Open	233	TECH-DEL-1584	15:10:03
LIVE-261011150955-3159	P3	FDH damage	DEL	Open	110	TECH-DEL-0679	15:09:55

8 rows · 264.9 ms round trip from client · captured live from the Lakebase endpoint during testing

Figure 5. Live incidents written by the operations feed every few seconds, with assigned technicians. A database trigger notifies the app on every change.

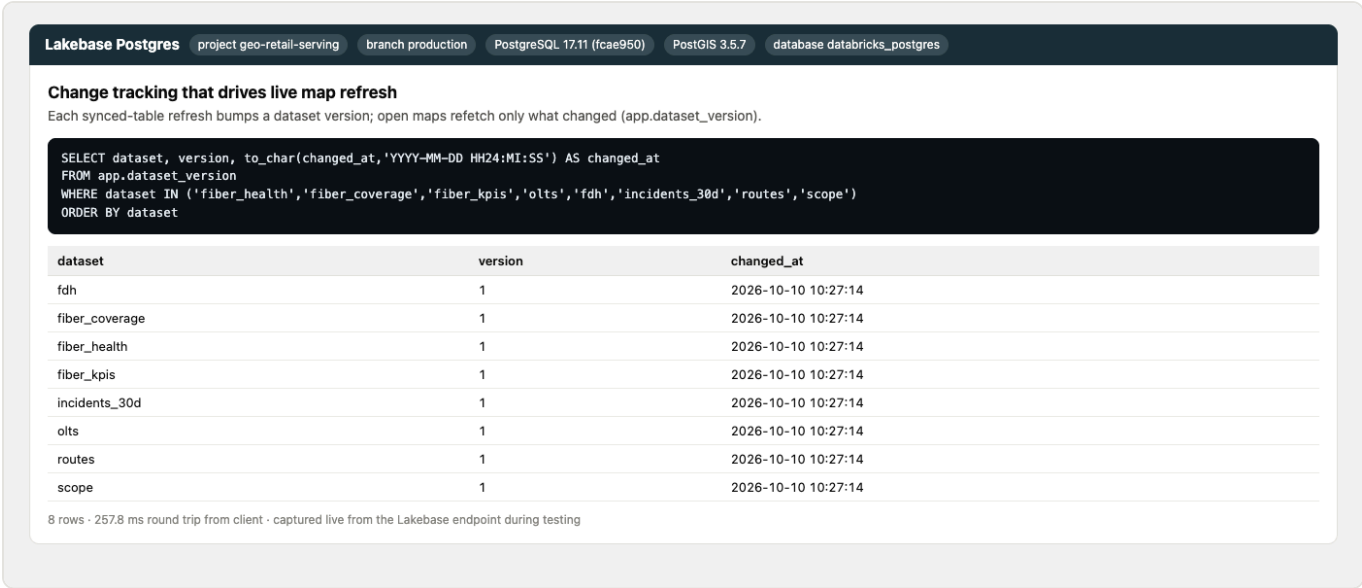


Figure 6. Dataset versions. When a synced table refreshes, its dataset version increases and every open map refetches only that layer.

6. Security and access control

Access is defined once, in a Unity Catalog table that maps each person or role to a scope: national, one or more regions, or one or more metros. That single definition is enforced in two places.

- **Unity Catalog row filters** on all 17 gold tables. Analysts using SQL, notebooks, dashboards or Genie only see rows in their scope.
- **Postgres row-level security in Lakebase.** The app reads through security-barrier views that apply the same scope to every query. The scope is resolved once per query, so enforcement adds no measurable latency.
- **Least privilege for the app.** The app's identity can read only the secured views and write only to its own operational tables. It cannot bypass row-level security.
- **Workspace single sign-on.** Users reach the app with their Databricks identity. Presenters can switch to demonstration personas to show what each role sees.

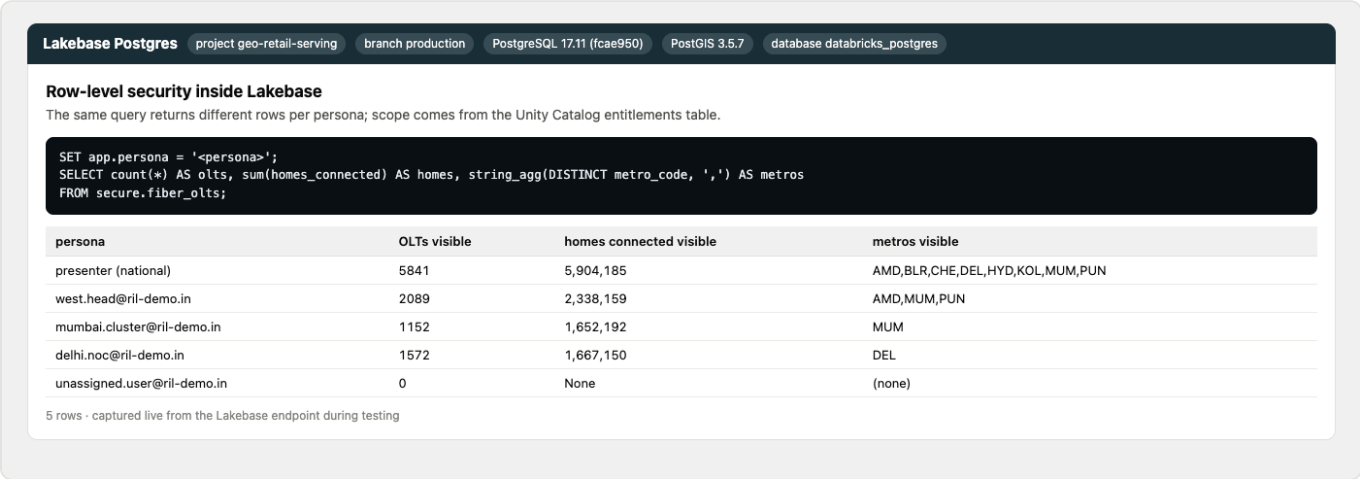


Figure 7. The same query in Lakebase returns different rows for each persona. A user with no entitlement sees nothing.

Persona	Scope	OLTs visible in Lakebase	OLTs visible via Unity Catalog filter
National view	All India	5,841	5,841
Regional Head, West	West region	2,089	2,089
Cluster Manager, Mumbai MMR	Mumbai metro	1,152	1,152
NOC Lead, Delhi NCR	Delhi metro	1,572	1,572
Ops Lead, South + Mumbai	South region and Mumbai metro	2,773	2,773
User with no entitlement	None	0	0

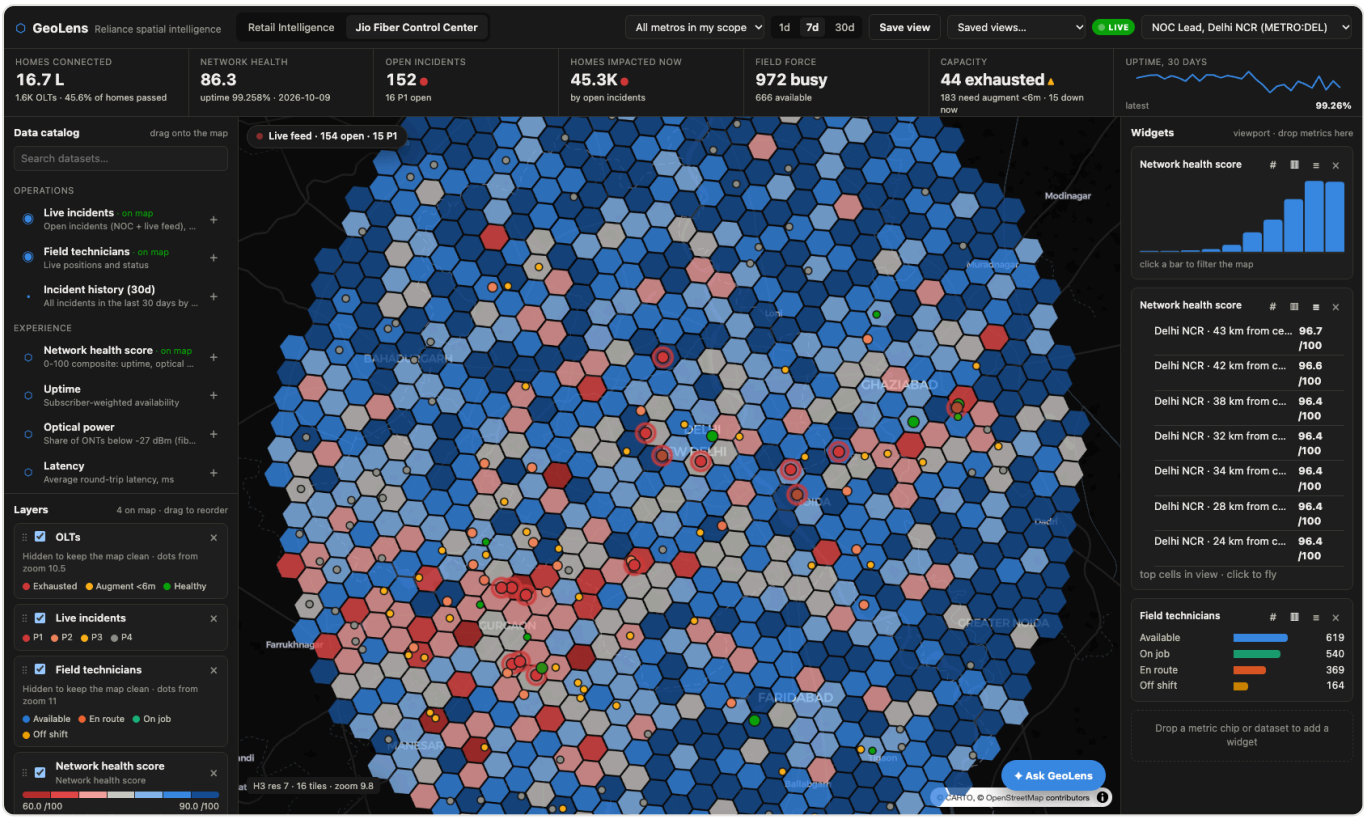


Figure 8. The same app viewed as the NOC Lead for Delhi NCR. The metro list, KPIs, map and widgets all contain Delhi only.

7. The storyline: a day in the NOC

This walkthrough follows a NOC lead through a typical session. Every screenshot was captured from the deployed app during the end-to-end test run described in section 9.

1 Land on what matters: live incidents

The control center opens on the national view with only live incidents switched on, so nothing distracts from what is broken right now. Each dot is an open incident coloured by severity. Critical P1 incidents glow continuously until they are fixed. The KPI strip shows homes connected, network health, open incidents, homes impacted, field force and capacity at a glance. Network health and technicians are already in the layer list, switched off, ready when needed.

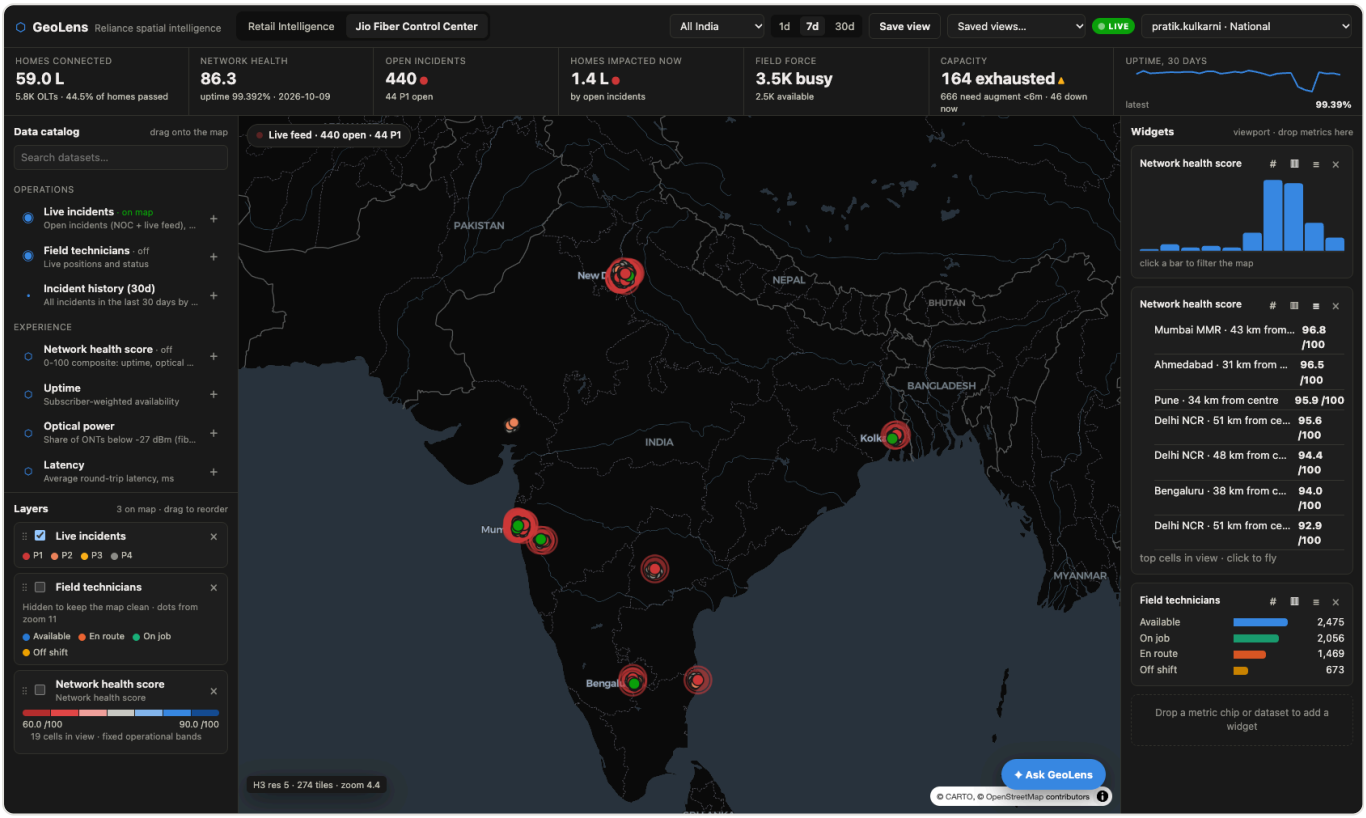


Figure 9. Landing view: open incidents only, with P1 incidents glowing.

2 Watch the live feed and triage

The live feed lists the newest incidents as they arrive. Clicking one flies the map to it and opens the incident panel: type, severity, homes impacted, start time, root cause and the five nearest available technicians with their skills and distance.

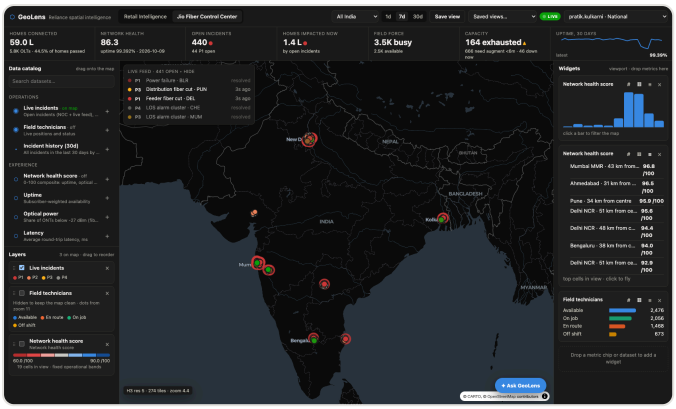


Figure 10. Live feed of incoming incidents.

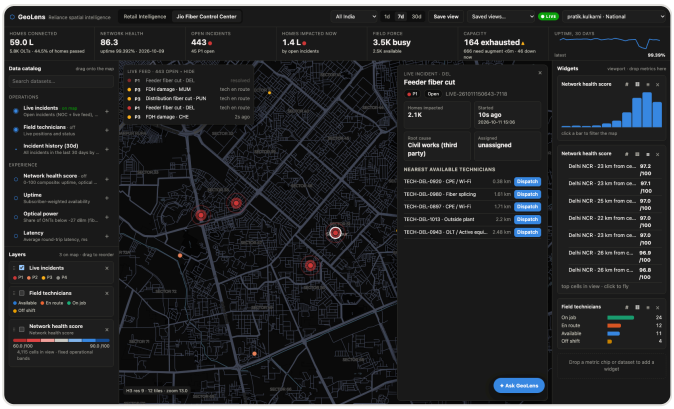


Figure 11. Incident panel with nearest available technicians.

3 Dispatch in one click

Dispatch assigns the chosen technician. The assignment is written to Lakebase, every connected screen is notified, and the technician starts moving towards the job on the map.

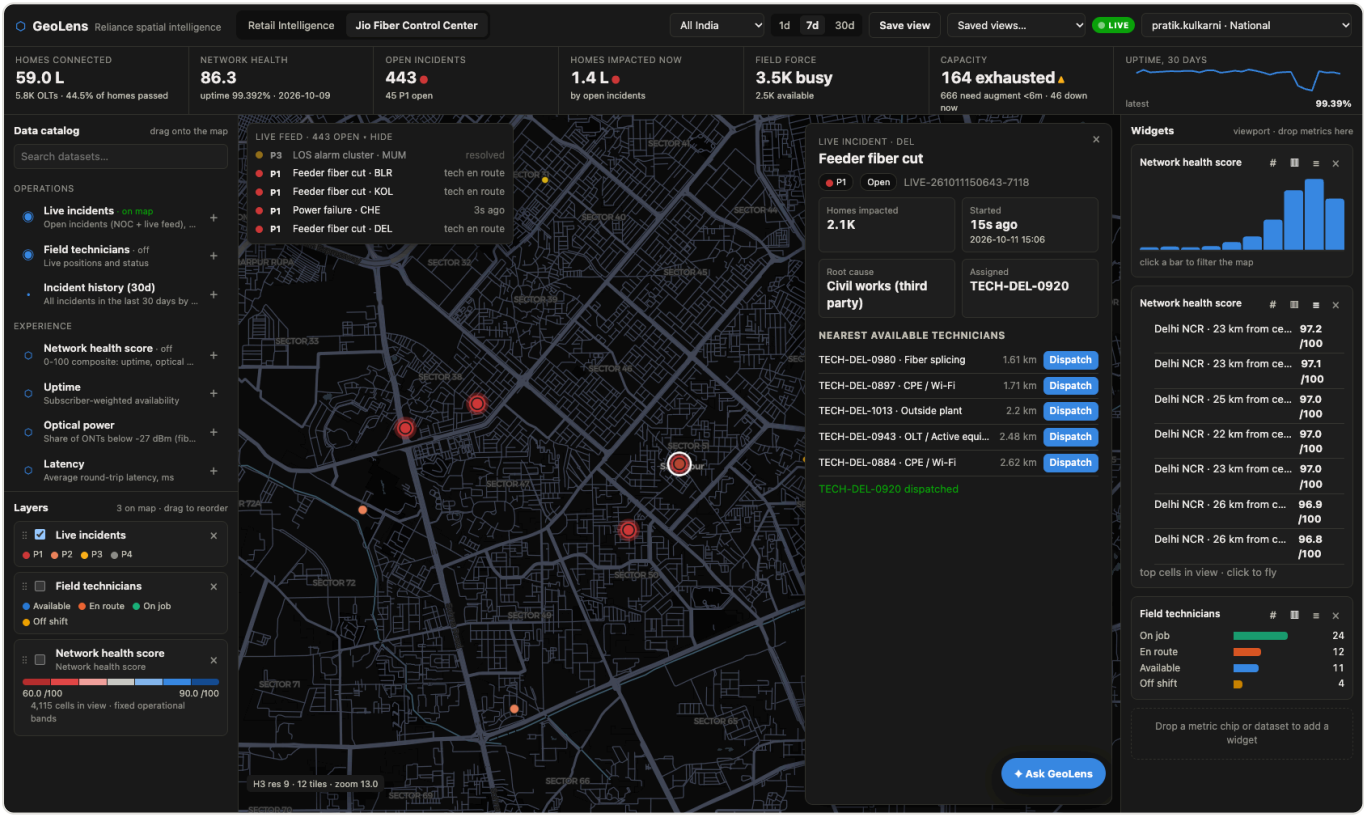


Figure 12. Technician dispatched; the incident now shows the assignment.

4 Turn on network health across India

Ticking Network health score adds customer experience to the map. At national zoom, cells are rolled up into larger hexagons so each metro stays readable. Colours use fixed operational bands (red below 60, blue above 90), so a red area means a real problem, not just the worst of a good set.

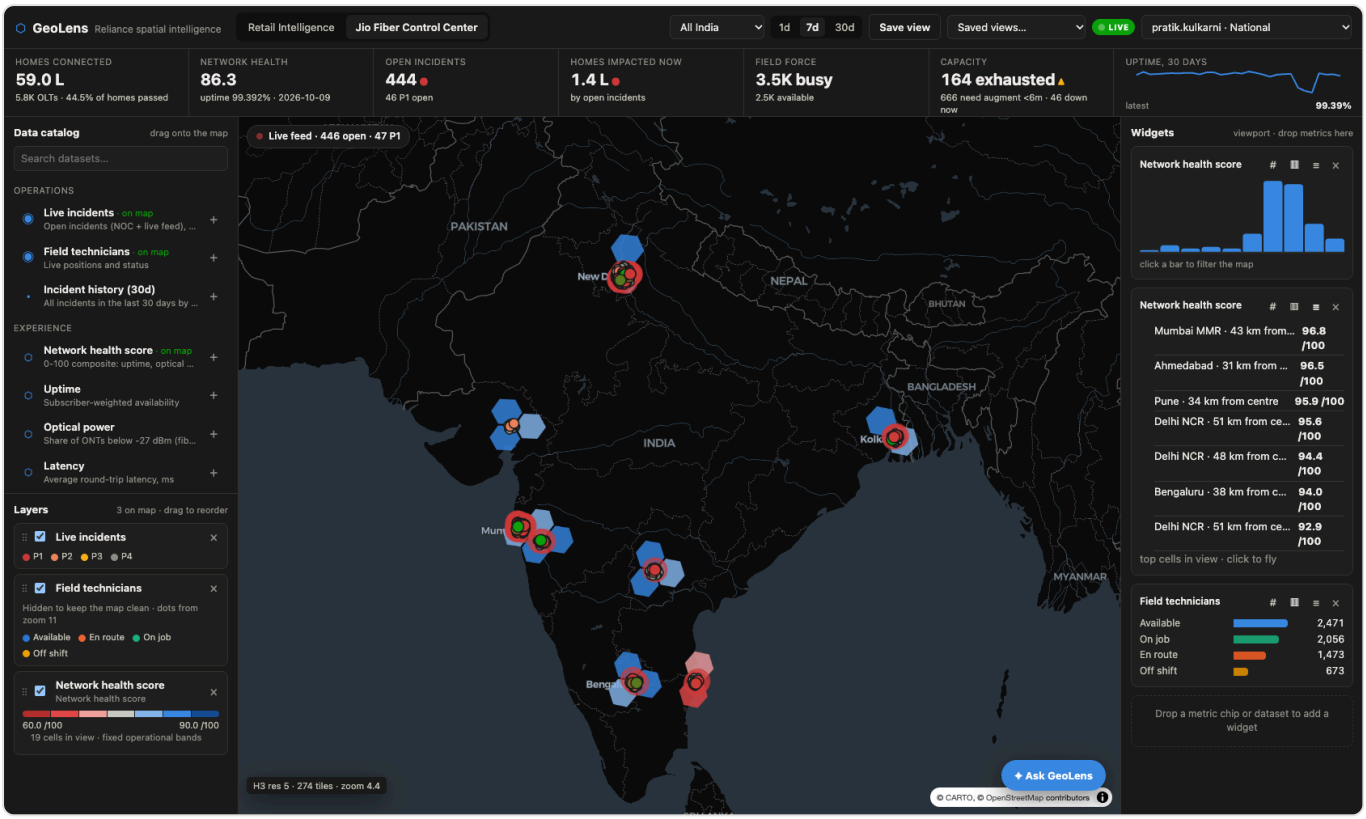


Figure 13. National view with network health and technicians switched on.

5 Drill into a hexagon without losing your place

Clicking a hexagon zooms to exactly that area and shows only the detail inside it. A breadcrumb records the path, and Back returns to the previous view.

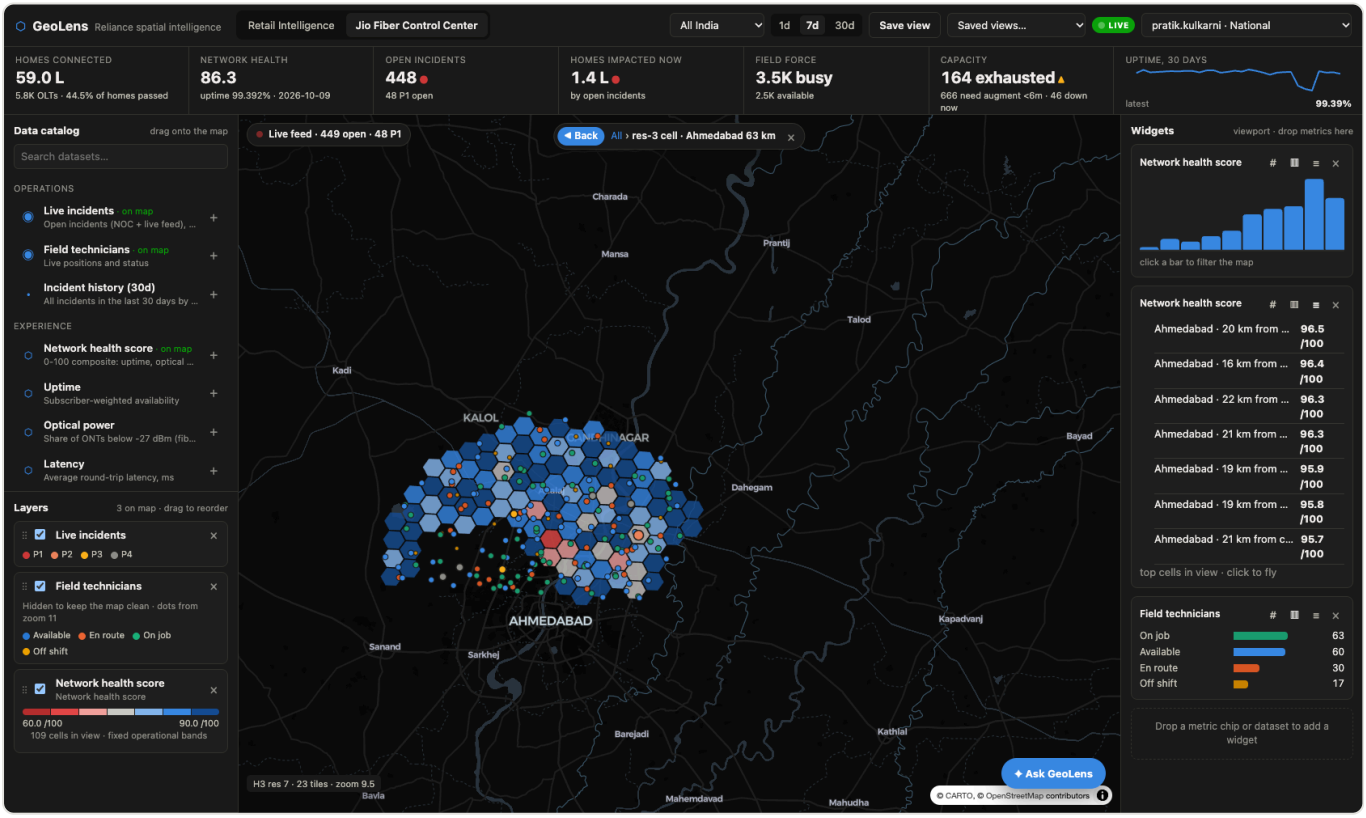


Figure 14. One click on a national hexagon drills into Ahmedabad, showing only the cells, incidents and technicians inside it.

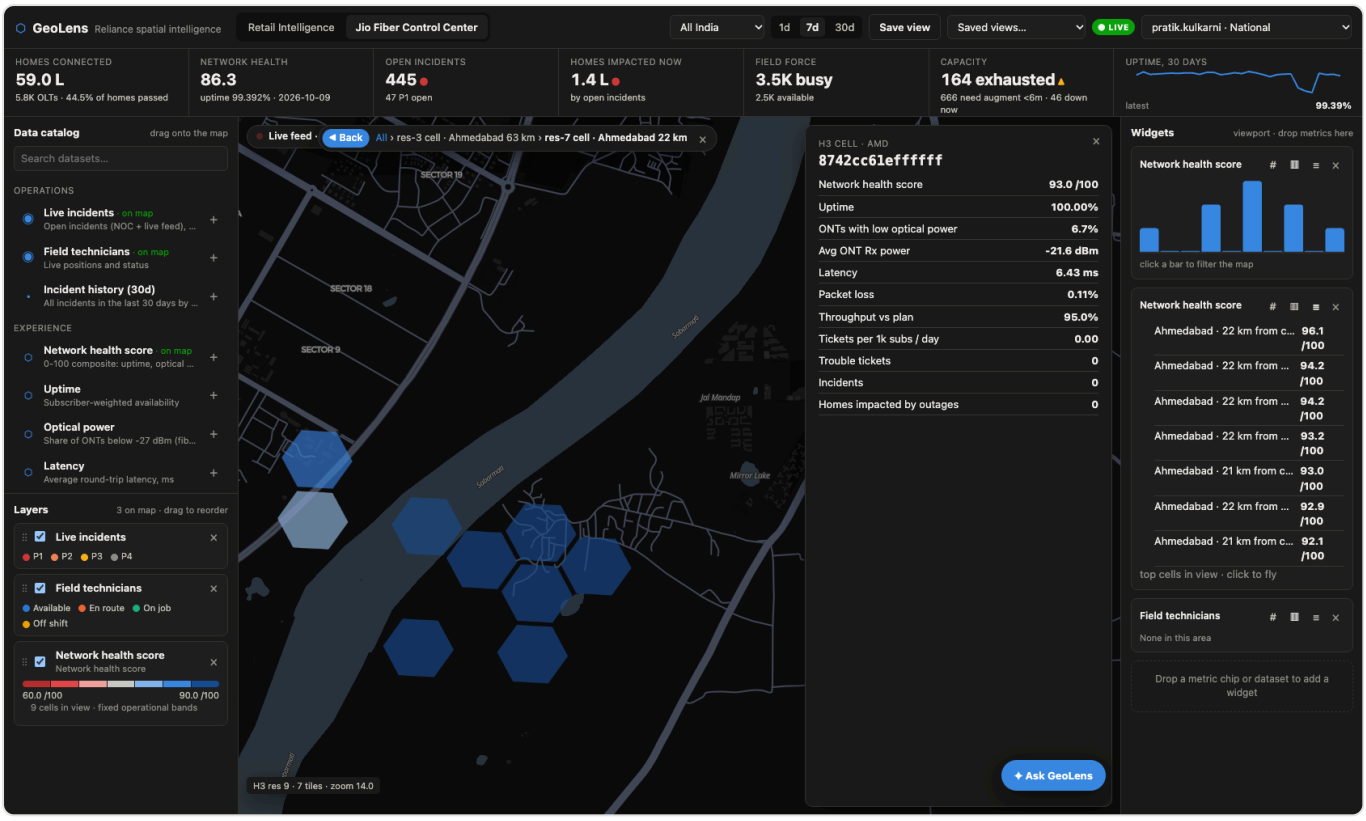


Figure 15. A second click goes one level deeper and opens the cell's full metrics: health, uptime, optical power, latency, loss, throughput, tickets and outage impact.

6 Focus on one metro

Choosing a metro in the top bar restricts everything on screen to that metro: map, KPIs, widgets, live feed and the AI assistant. The histogram and top-cells widgets recompute for the area in view, and a click on any bar filters the map.

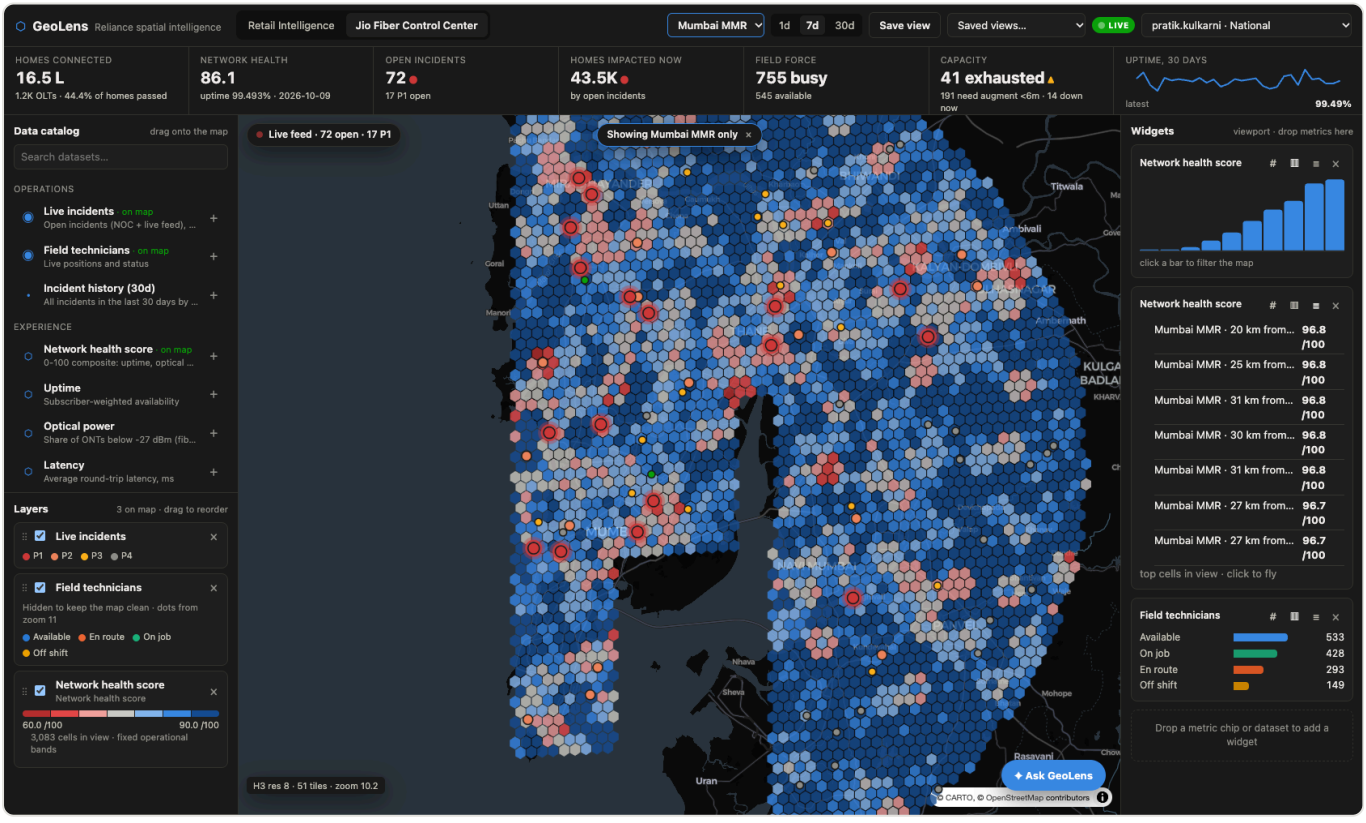


Figure 16. Mumbai MMR only: health hexagons, live incidents and technicians, with widgets scoped to Mumbai.

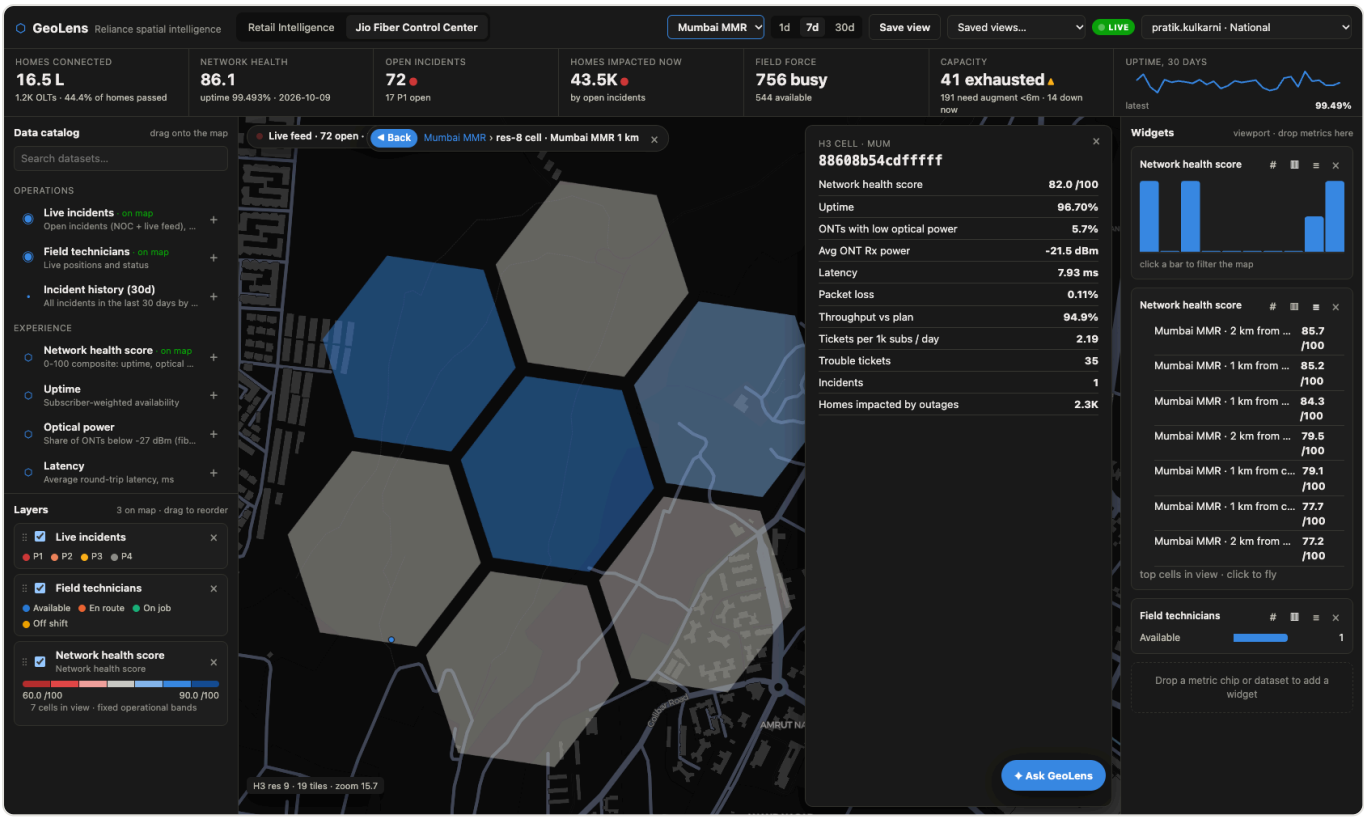


Figure 17. Street-level drill in Mumbai: a single neighbourhood cell with its child hexagons and full metrics.

7 Ask GeoLens

The assistant answers operational questions in plain language. It queries the same governed data the map uses, respects the user's scope and the selected metro, and acts on the map: it adds layers, flies to places, highlights cells and opens the relevant OLT.



Figure 18. "Which OLTs will run out of capacity first and where are they?" The assistant lists the exhausted Mumbai OLTs with PON fill and peak utilisation, adds the OLT layer and opens the most critical one.

8 Build your own view with drag and drop

The data catalog on the left groups 17 layers into Operations, Experience, Growth and Network. Users drag a layer onto the map or into the widget rail, reorder layers by dragging, change the metric, colour and size, and save the view to share with colleagues. No SQL or code is needed.

Group	Layers available
Operations	Live incidents, field technicians, incident history (30 days)
Experience	Network health score, uptime, optical power, latency, trouble tickets, outage impact
Growth	Penetration, sales opportunity (passed, not connected), build-out opportunity, premium plan mix, Reliance Retail cross-sell
Network	OLTs by capacity status, FDH cabinets by health, fiber routes with cuts highlighted

9 See only what you are entitled to

Switching to the Delhi NOC persona (Figure 8) shows the entitlement model at work: the same app, the same map, but only Delhi data, enforced in the database rather than in the screen.

8. Always current: data freshness

Two kinds of change reach the map automatically, with no page reload.

- **Live operations** (new incidents, dispatches, technician movement, resolutions) are written to Lakebase and pushed to every open browser within seconds through Postgres notifications and server-sent events.
- **Analytical refreshes** (new daily KPIs, updated OLT inventory, new incidents in the lakehouse) arrive through synced tables. The app detects the new table version, increases the dataset version and tells every open map to refetch only the affected layers.

Most of this time is the synced-table refresh itself. Refreshes can be scheduled or triggered when the upstream gold table changes; live operational data does not wait for them.

Measured during testing (OLT inventory change)	Run 1	Run 2
Update written to the Delta table in Unity Catalog	0 s	0 s
Synced table refresh triggered	6 s	5 s
Change detected; "dataset changed" event pushed to all open maps	32 s	56 s
App serving the new value	33 s	58 s
Open browser updated its widgets after the event (no reload)	1 to 2 s	1 to 2 s

9. End-to-end test results

The deployed solution was tested end to end on 11 October 2026: platform, API, security, data parity, live updates, freshness and the full browser storyline. Browser tests used real mouse clicks in a headless Chrome session against the production app URL.

Area	What was checked	Result
Platform	App running on 4 instances; Lakebase primary and read replica active; PostgreSQL 17 with PostGIS 3.5	**Pass**
API	Every route: metadata, catalog, H3 tiles, ETag revalidation (304), points, vector tiles, KPIs, OLT and incident detail, live feed (full and delta), dispatch, saved views, health, event stream	**Pass (all routes)**
Postgres row-level security	Five personas plus a user with no entitlement, on OLTs, stores, health cells and live tables	**Pass**
Unity Catalog governance	Row filters attached to all 17 gold tables; filter logic per persona returns exactly the Lakebase counts	**Pass**
Data parity	Row counts identical in gold Delta, the serving copy and Lakebase (for example 5,225,190 health rows, 5,841 OLTs, 25,605 incidents)	**Pass**
Live operations	Simulator leader active; incidents and technician positions updated within seconds; live events received over the stream	**Pass**
Freshness	Delta change to app in 33 to 58 s across runs; open browser updated without reload	**Pass**
Browser storyline	13 scripted steps (Section 7): landing, live feed, incident, dispatch, national health, two-level drill, Back, metro focus, street-level cell, AI assistant, persona	**Pass, 13 of 13**
Browser health	Console errors and failed app requests across the full walkthrough (about 790 API requests)	**0 errors, 0 failures**

Improvements made as a result of testing

Observation	Change
Health hexagons were too small to see at national zoom	Cells now roll up into larger hexagons when zoomed out (sums for counts, averages for rates), and remain clickable
Drilling a national hexagon framed a mostly empty area	Drill now fits the area that actually contains data
The breadcrumb ran under the details panel	Breadcrumb now centres on the visible part of the map
The assistant panel covered the details panel	The assistant now opens beside the details panel
Catalog marked switched-off layers as "on map"	Catalog now distinguishes "on map" from "off", and adding a switched-off layer turns it on
Empty widgets after drilling into a quiet area	Widgets now say "None in this area"
P1 glow was faint at metro zoom	Glow and ripple made stronger and consistent at every zoom

10. Scale and performance

Load tests ran from a Databricks serverless job in the same cloud region as the app, simulating map users who pan, zoom and refresh with HTTP/2 connections, as browsers do.

Scenario	Throughput	Errors	Server time per tile
150 users, no think time (saturation)	299 req/s	0	p50 12.8 ms · p95 35 ms
1,000 users on 1 app instance	206 req/s	0.07%	p50 18 ms
1,000 users on 4 app instances	182 req/s	connection limits	p50 24 ms

- **The data layer is not the bottleneck.** Lakebase and the app answer each map tile in tens of milliseconds even under saturation, thanks to tile-sized queries, a shared cache with version-aware keys, ETag revalidation and the read replica.
- **Single-source load tests hit an ingress limit.** All simulated users came from one machine and one identity. Throughput levelled off at 200 to 300 requests per second regardless of instance count, and most client-side wait happened before requests reached the app. Real users arrive from many devices and identities and are not limited this way.
- **Validating thousands of real users** calls for a distributed, multi-identity test. The architecture scales horizontally on both tiers: more app instances behind the same URL, and more Lakebase read replicas for map traffic.

Why it stays fast

- Each map request covers one fixed tile of at most 343 hexagons, so query cost is bounded regardless of zoom.
- Entitlement scope is evaluated once per query instead of once per row.
- Responses are cached per scope and dataset version. When data changes, the version changes and stale entries are never served.
- Browsers revalidate with ETags, so unchanged tiles return in a few hundred bytes.

Appendix A. Metric definitions

Metric	Definition	Better when
Network health score	0 to 100 composite of uptime, optical margin, latency, packet loss, throughput against plan and trouble tickets, weighted by subscribers	Higher
Uptime	Subscriber-weighted availability over the selected period	Higher
ONTs with low optical power	Share of ONTs receiving below -27 dBm, an early sign of fiber degradation	Lower
Average ONT receive power	Mean optical receive power at the customer terminal, in dBm	Higher
Latency	Average round-trip latency, in milliseconds	Lower
Packet loss	Share of packets lost	Lower
Throughput against plan	Delivered speed as a share of the subscribed plan speed	Higher
Tickets per 1,000 subscribers per day	Trouble ticket rate normalised by subscriber base	Lower
Homes impacted by outages	Homes affected by incidents in the period	Lower
PON fill	Connected homes as a share of PON port capacity on an OLT	Below threshold
Capacity status	Exhausted, Augment within six months, or Healthy, from PON fill, peak utilisation and growth	Healthy
Penetration	Homes connected as a share of homes passed	Higher
Sales opportunity	Homes passed but not connected	Context
Retail cross-sell	Reliance Retail customers in the area without Jio Fiber	Context

Appendix B. Glossary

H3	A global grid of hexagonal cells at multiple resolutions. Resolution 5 cells are about 250 km², resolution 9 cells about 0.1 km².
OLT	Optical Line Terminal, the exchange-side equipment that serves many homes over passive optical splitters.
PON	Passive Optical Network, the shared fiber tree from an OLT port to homes.
FDH	Fiber Distribution Hub, the street cabinet that holds splitters.
ONT	Optical Network Terminal, the device in the customer's home.
Lakebase	Databricks managed Postgres, integrated with Unity Catalog and able to sync tables from the lakehouse.
Synced table	A Lakebase table kept in step with a Unity Catalog Delta table without custom pipelines.
Row filter / RLS	Database rules that return only the rows a user is entitled to see, applied by Unity Catalog and by Postgres row-level security.
Databricks App	A web application hosted on Databricks with workspace sign-on and direct, governed access to platform data and AI.

Appendix C. Technology stack

GeoLens is built on Databricks platform services for data, governance, serving, hosting and AI, with permissively licensed open-source libraries for the map experience.

Databricks platform

Layer	Technology	Role in GeoLens
Storage and processing	Delta Lake on Unity Catalog (bronze, silver, gold)	Source data, cleaning, H3 indexing and KPI rollups at H3 resolutions 5 to 9
Compute	Serverless SQL warehouse	Builds the data model using built-in H3 and spatial SQL functions
Governance	Unity Catalog	Row filters on 17 gold tables, one entitlements table for region and metro scope, documented tables and columns
Serving database	Lakebase (managed PostgreSQL 17 with PostGIS 3.5), primary and read replica	Millisecond map queries, vector tiles for fiber routes, row-level security, live operational tables
Lakehouse to Lakebase	Synced tables	Keeps 18 gold tables in step with Postgres without custom pipelines
Application hosting	Databricks Apps	Hosts the API and web interface with workspace single sign-on and a least-privilege service identity
AI	Foundation Model APIs through AI Gateway	Powers the Ask GeoLens assistant, which queries governed data and drives the map
Testing and operations	Lakeflow Jobs (serverless)	In-region load testing

Application backend

Component	Technology
API framework	Python with FastAPI and Uvicorn
Database access	psycopg2 with a pooled, automatically refreshed OAuth connection to Lakebase
Live updates	PostgreSQL LISTEN/NOTIFY, delivered to browsers as server-sent events
Change detection	A single elected background worker detects synced-table refreshes and advances dataset versions
Performance	Shared caches keyed by scope and dataset version, ETag revalidation, compression, server timing metrics

Map and user interface

Component	Technology
Web framework	React 18, built with Vite
Map visualisation	deck.gl 9 (hexagon, point, vector tile and line layers)
Map engine	MapLibre GL, integrated through react-map-gl
Hexagon grid	H3 (h3-js) for roll-ups, drill-down and cell geometry
Basemap	Dark Matter and Positron vector basemap styles over OpenStreetMap data, currently served from a public basemap CDN; can be replaced by a basemap self-hosted inside Databricks
Charts and widgets	Lightweight custom SVG components with colourblind-safe palettes

Open-source licences

All non-Databricks components are permissively licensed: React and deck.gl (MIT), MapLibre GL (BSD), H3 (Apache 2.0), PostgreSQL and PostGIS (PostgreSQL licence and GPL for the PostGIS server extension, used as a managed service). Map data is from OpenStreetMap (ODbL), credited on the map. The only external service in the current build is the basemap CDN.

Appendix D. Basemap: removing the CARTO dependency

GeoLens was built to run entirely on Databricks and open-source software. In the current build there is one remaining third-party dependency: the background basemap is loaded from CARTO's public basemap service. This appendix explains exactly what that dependency covers and how it is removed.

What depends on CARTO today

Item	Today	Effect
Basemap styles	Dark Matter (Jio Fiber view) and Positron (light view), loaded from `basemaps.cartocdn.com`	The grey or dark background map: roads, water, land use, place names and borders
Basemap tiles	Vector tiles served by CARTO's content delivery network	Each user's browser downloads background tiles directly from CARTO
Fonts and map icons	Label fonts and sprites referenced by the CARTO styles	Used to draw place names on the background map
Attribution	"© CARTO, © OpenStreetMap contributors" shown on the map	Required while CARTO's service is used

What does not depend on CARTO

- All Jio Fiber data, hexagons, incidents, technicians, OLTs, fiber routes, KPIs and widgets, which are served from Lakebase by the GeoLens app.
- Security, entitlements, live updates and the AI assistant, which run on Unity Catalog, Lakebase, Databricks Apps and Foundation Model APIs.
- The map software itself: deck.gl, MapLibre GL and H3 are open source and need no CARTO account, key, SDK or licence.

No CARTO account, API key, software development kit or paid service is used anywhere in GeoLens.

Why remove it

- **Licensing.** CARTO's free basemaps are intended for limited use. Production use at Jio scale would require a commercial agreement.
- **Network access.** Users' browsers must reach an external service on the internet, which may not be allowed inside a restricted enterprise network.
- **Availability and control.** The look and uptime of the background map depend on a third party.

Open alternatives considered

Option	Look compared with today	Where it runs	Licence
OpenMapTiles data with the open Dark Matter and Positron styles	Nearly identical; same colours, fonts and layering	Inside Databricks	Open (BSD, CC-BY 4.0, ODbL)
Protomaps with its own light and dark themes	Similar minimal feel, different palette and labels	Inside Databricks	Open (BSD, ODbL)
OpenFreeMap public service	Very close	Volunteer-run public servers	Open, but an external dependency
Commercial basemap providers	Close	Provider cloud	Paid licence

The Dark Matter and Positron styles are themselves open source, published through the OpenMapTiles project. Keeping them preserves the look users already know, while the tiles, fonts and icons move inside the Databricks environment.

How the dependency is removed

Step	What happens
1. Build the basemap	Generate OpenStreetMap vector tiles in the OpenMapTiles schema for India, with full street detail for the served metros, using the open-source Planetiler tool on Databricks serverless compute.
2. Store it in Databricks	Keep the tile archive in a Unity Catalog Volume and load the tiles into Lakebase, so the basemap is served the same governed, low-latency way as the network data.
3. Serve it from GeoLens	Add a basemap tile endpoint to the GeoLens app with caching and revalidation, and bundle the open label fonts and map icons with the application.
4. Switch the map	Point the map engine at self-hosted copies of the Dark Matter and Positron styles and update the on-map attribution.
5. Verify	Rerun the end-to-end storyline and confirm the browser makes no requests to any external map service.
6. Keep it current	Rebuild the basemap on a schedule with Lakeflow Jobs to pick up new roads and places from OpenStreetMap.

After the change

- No CARTO service, content delivery network, account or licence is used.
- The application works inside a closed network: every map request goes to the GeoLens app URL.
- The map keeps the same look and feel, with small differences in label density at some zoom levels.
- The only remaining obligation is attribution on the map: "© OpenMapTiles © OpenStreetMap contributors", with credit for the open style design as its licence requires.